BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division:C Batch: 2 Practical No: 2

Roll no: 67 PRN No: 12415134 Name of Student: Surabhi Patel

Subject Name : Deep Learning

Problem Statement : Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset, and evaluate its performance using accuracy and a confusion matrix

Algorithm :

- **Start** the program.
- **Import** the required libraries for data handling, visualization, preprocessing, model building, and evaluation.
- **Load** the Iris dataset using `load_iris()`.
- **Separate** the dataset into:
    - Input features (`X`)
    - Target labels (`y`)
- **Visualize** the dataset using a scatter plot to observe the distribution of different Iris flower species.
- **Split** the dataset into training and testing sets using `train_test_split()`.
- **Standardize** the feature values using `StandardScaler()` to improve model performance.
- **Create** an Artificial Neural Network (ANN) model using `MLPClassifier` with:
    - Two hidden layers of 10 neurons each
    - ReLU activation function
    - Adam optimizer
    - Maximum 1000 training iterations
- **Train** the ANN model using the training dataset with the `fit()` method.
- **Predict** the class labels for the testing dataset using the `predict()` method.
- **Evaluate** the model by calculating:

- Accuracy
- Confusion Matrix
- Classification Report (Precision, Recall, and F1-Score)

- **Visualize** the Confusion Matrix to analyze the classification performance.
- **Plot** the Training Loss Curve to observe how the loss decreases during model training.
- **Display** all results and visualizations.
- **Stop** the program.

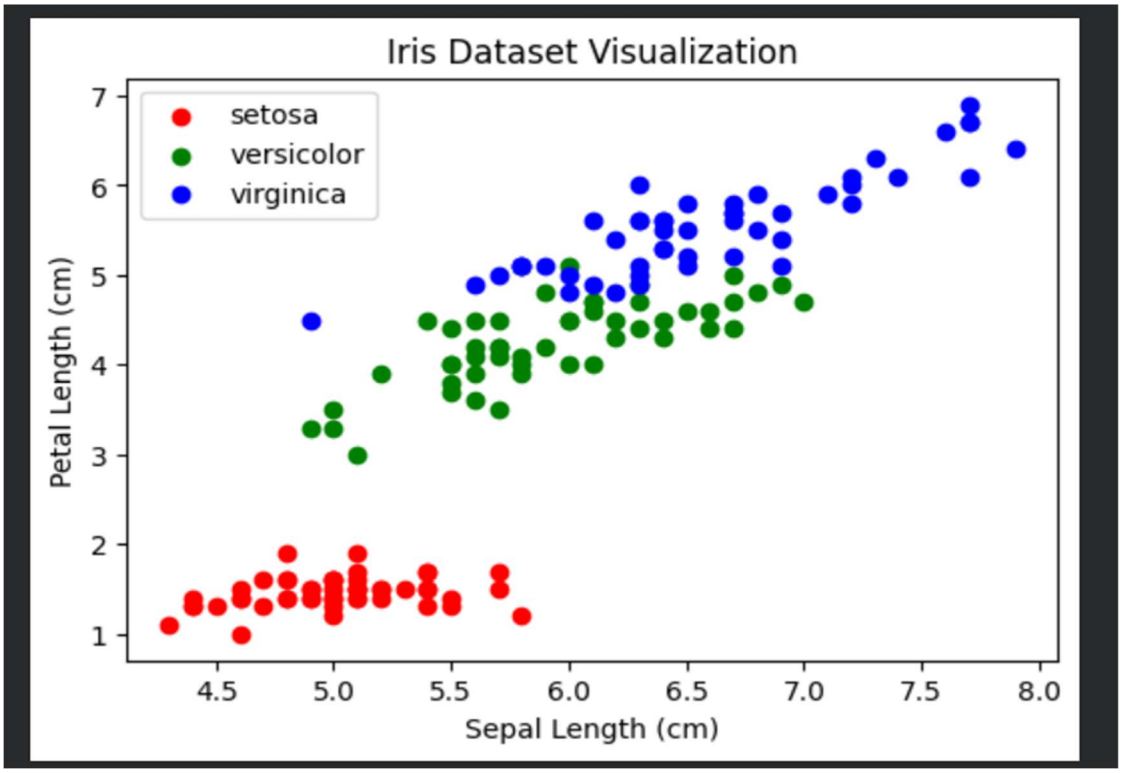
Code :

```
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report, ConfusionMatrixDisplay

# Load Dataset
iris = load_iris()
X, y = iris.data, iris.target

# Data Visualization

df = pd.DataFrame(X, columns=iris.feature_names)
df["Species"] = y

plt.figure(figsize=(6,4))

colors = ['red', 'green', 'blue']

for i in range(3):
    plt.scatter(
        df[df["Species"] == i]["sepal length (cm)"],
        df[df["Species"] == i]["petal length (cm)"],
        color=colors[i],
        label=iris.target_names[i]
    )

plt.title("Iris Dataset Visualization")
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Petal Length (cm)")
plt.legend()
```

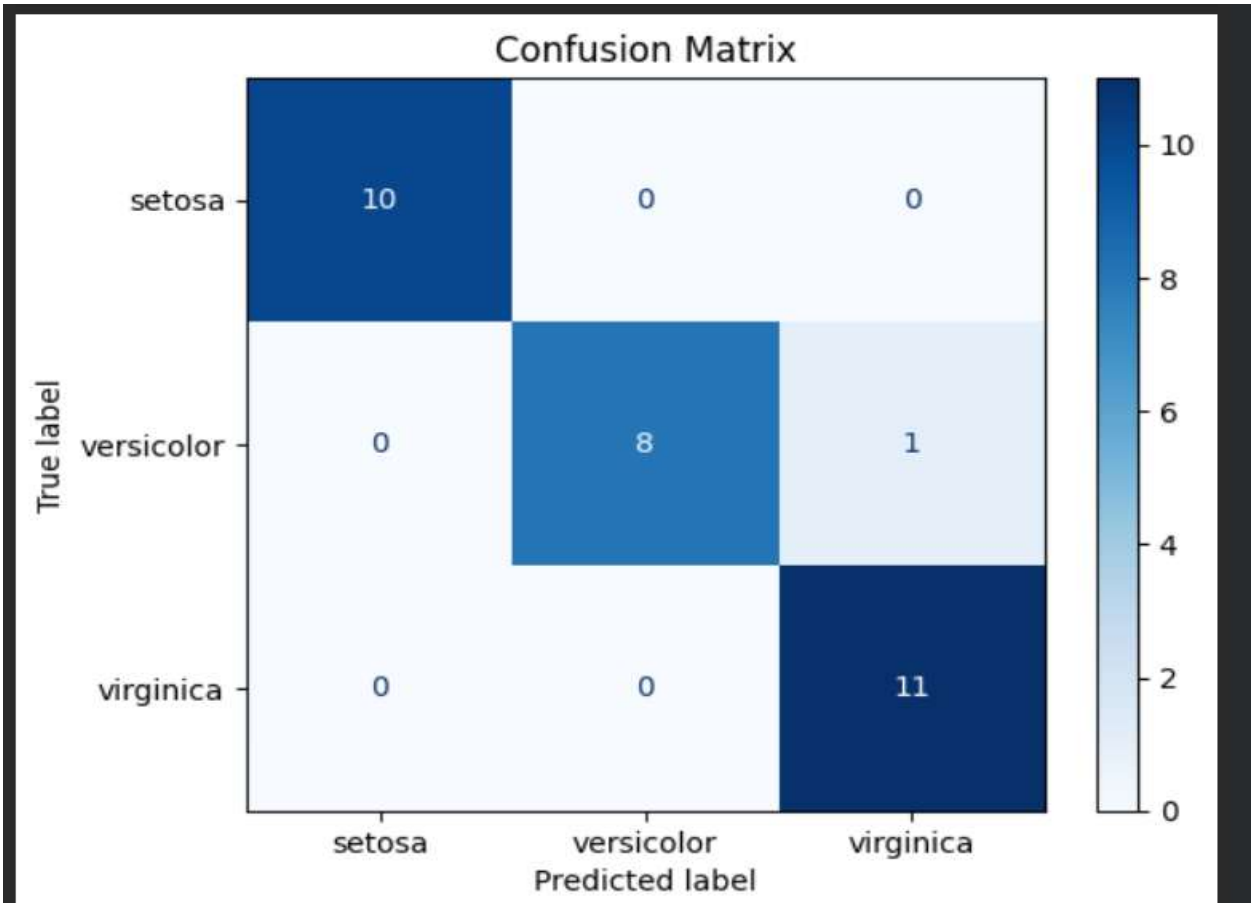
```
plt.show()

# Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Feature Scaling

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ANN Model

mlp = MLPClassifier(
    hidden_layer_sizes=(10,10),
    activation='relu',
    solver='adam',
    max_iter=1000,
    random_state=42
)

# Train Model & prediction
mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

# Evaluation

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Confusion Matrix Visualization

ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred,
    display_labels=iris.target_names,
    cmap="Blues"
)

plt.title("Confusion Matrix")
plt.show()
```

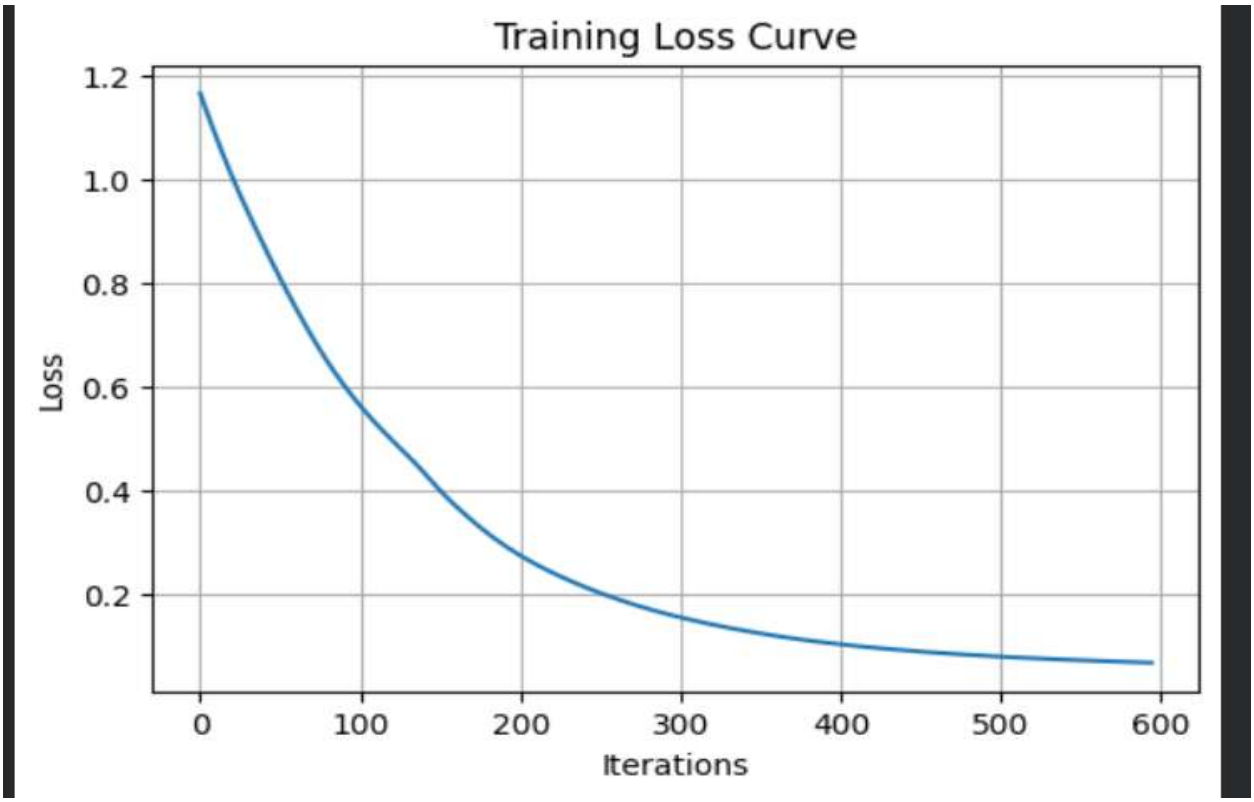
```
# Loss Curve

plt.figure(figsize=(6,4))
plt.plot(mlp.loss_curve_)
plt.title("Training Loss Curve")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.grid(True)
plt.show()
```

Output :




```
Accuracy: 0.9666666666666667
Confusion Matrix:
 [[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      0.89      0.94         9
           2       0.92      1.00      0.96        11

    accuracy                           0.97        30
   macro avg       0.97      0.96      0.97        30
weighted avg       0.97      0.97      0.97        30
```


Confusion Matrix
setosa
versicolor
virginica
True label
10
0
0
0
8
1
0
0
11
setosa
versicolor
virginica
Predicted label
10
8
6
4
2
0


Training Loss Curve
Loss
1.2
1.0
0.8
0.6
0.4
0.2
0
100
200
300
400
500
600
Iterations